

Passagem de parâmetro por referencia

`melise@unifei.edu.br`

Passagem de Parâmetro em C

- Em C, há duas maneiras de fazer a passagem de parâmetro. Já vimos funções com os parâmetros passados por valor. Nesse caso, o valor de uma variável declarada em uma função é copiado para o parâmetro de outra função.
- No exemplo ao lado, os valores da variável x e y da main são copiados para os parâmetros x e y da função troca. Dessa forma, quando chamamos a função troca na main, os valores de x e y da main não mudam. Somente x e y da função troca são alterados.

```
#include <stdio.h>

void troca(int x, int y){
    int aux;
    aux = x;
    x = y;
    y= aux;
    return;
}

int main(void) {
    int x = 10, y=20;
    printf("x = %d e y = %d", x, y);
    troca(x,y);
    printf("\nx = %d e y = %d", x, y);
    return 0;
}
```

Passagem de Parâmetro em C

- Outra forma de fazer a passagem de parâmetro em C é por referência, ou seja, os parâmetros da função chamada são referências às variáveis da função chamadora. Em C, uma referência é um ponteiro, ou seja, uma variável que guarda um endereço.
- Logo, quando passamos um ponteiro como parâmetro em uma função em C, estamos fazendo a passagem por referência. Mas quando usar essa estratégia?

Passagem de Parâmetro em C

- Imagine que você quer trocar os valores de x e y, declarados na função main, lá dentro da função troca. Só há um jeito de fazer isso, passando o endereço de x e o endereço de y para a função troca. Com esse endereço, a função troca consegue fazer essa alteração.
- Vamos mudando o código passo a passo, vamos primeiro mudar a chamada da função troca na main

A função troca agora recebe o endereço da variável x e o endereço da variável y

```
#include <stdio.h>
void troca(int x, int y){
    int aux;
    aux = x;
    x = y;
    y= aux;
    return;
}
```

```
int main(void) {
    int x = 10, y=20;
    printf("x = %d e y = %d", x, y);
    troca(&x,&y);
    printf("\nx = %d e y = %d", x, y);
    return 0;
}
```

Passagem de Parâmetro em C

- Bom, depois de mudar a chamada da função troca na main, temos que adaptar o cabeçalho função pois os parâmetros da função não são mais dois inteiros, são dois ponteiros para inteiros

```
#include <stdio.h>
void troca(int *px, int *py){
    int aux;
    aux = *x;
    *x = *y;
    *y = aux;
    return;
}
```

```
int main(void) {
    int x = 10, y=20;
    printf("x = %d e y = %d", x, y);
    troca(&x,&y);
    printf("\nx = %d e y = %d", x, y);
    return 0;
}
```

Passagem de Parâmetro em C

- O próximo passo é alterar o código da função troca, agora, px e py são ponteiros que apontam para dois valores inteiros.

```
void troca(int *px, int *py){  
    int aux;  
    aux = *px;  
    *px = *py;  
    *py= aux;  
    return;  
}
```

```
int main(void) {  
    int x = 10, y=20;  
    printf("x = %d e y = %d", x, y);  
    troca(&x,&y);  
    printf("\nx = %d e y = %d", x, y);  
    return 0;  
}
```

Passagem de Parâmetro em C

- Agora, temos as duas funções alteradas. Quando a função main chamar a função troca:
 - o endereço da variável x da main será atribuído ao ponteiro px da função troca
 - o endereço da variável y da main será atribuído ao ponteiro py da função troca
 - Quando *px = *py, o endereço apontado por px (endereço de x) vai receber o valor apontado por py (valor de y).
 - Quando *py = aux, o endereço apontado por py (endereço de y) vai receber o valor de aux
 - Quando a função for finalizada, os valores de x e y foram realmente trocados

```
#include <stdio.h>

void troca(int *px, int *py){
    int aux;
    aux = *px;
    *px = *py;
    *py= aux;
    return;
}

int main(void) {
    int x = 10, y=20;
    printf("\nx = %d e y = %d", x, y);
    troca(&x,&y);
    printf("\nx = %d e y = %d", x, y);
    return 0;
}
```

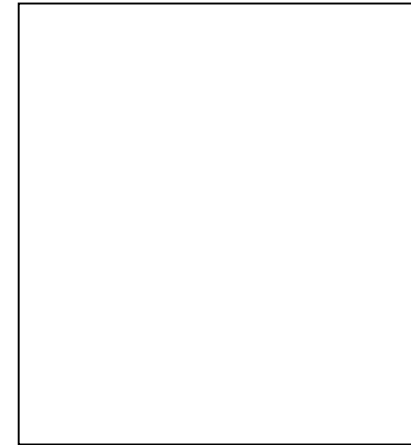
```
#include <stdio.h>
```

```
void troca(int *px, int *py){  
    int aux;  
    aux = *px;  
    *px = *py;  
    *py= aux;  
    return;  
}
```

Começa a execução do programa
na main

```
int main(void) {  
    int x = 10, y=20;  
    printf("\nx = %d e y = %d", x, y);  
    troca(&x,&y);  
    printf("\nx = %d e y = %d", x, y);  
    return 0;  
}
```

memória



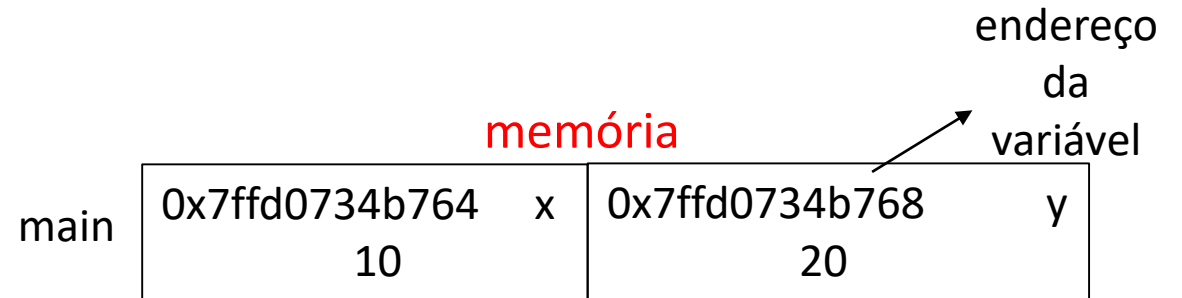
Essa é uma simplificação do que acontece na memória
Os endereços utilizados são fictícios

```
#include <stdio.h>
```

```
void troca(int *px, int *py){  
    int aux;  
    aux = *px;  
    *px = *py;  
    *py= aux;  
    return;  
}
```

```
int main(void) {  
    int x = 10, y=20;  
    printf("\nx = %d e y = %d", x, y);  
    troca(&x,&y);  
    printf("\nx = %d e y = %d", x, y);  
    return 0;  
}
```

As variáveis x e y são alocados na memória e os valores são atribuídos às variáveis

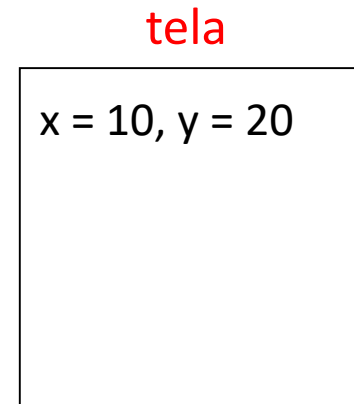
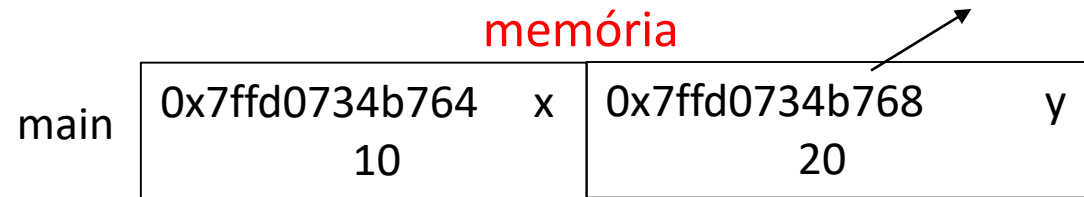


```
#include <stdio.h>
```

```
void troca(int *px, int *py){  
    int aux;  
    aux = *px;  
    *px = *py;  
    *py= aux;  
    return;  
}
```

```
int main(void) {  
    int x = 10, y=20;  
    printf("\nx = %d e y = %d", x, y);  
    troca(&x,&y);  
    printf("\nx = %d e y = %d", x, y);  
    return 0;  
}
```

Os valores são impressos na tela, nada muda na memória



Essa é uma simplificação do que acontece na memória

```
#include <stdio.h>
```

```
void troca(int *px, int *py){  
    int aux;  
    aux = *px;  
    *px = *py;  
    *py = aux;  
    return;  
}
```

```
int main(void) {  
    int x = 10, y=20;  
    printf("\nx = %d e y = %d", x, y);  
    troca(&x,&y);  
    printf("\nx = %d e y = %d", x, y);  
    return 0;  
}
```

A função troca é chamada
O controle sai da função
main e passa para a função
troca

memória

troca	0x7ffe1702e370 px	0x7ffe1702e378 py
main	0x7ffd0734b764 x 10	0x7ffd0734b768 y 20

Os parâmetros px e py são
alocados na memória

tela

x = 10, y = 20

```
#include <stdio.h>
```

```
void troca(int *px, int *py){  
    int aux;  
    aux = *px;  
    *px = *py;  
    *py= aux;  
    return;  
}
```

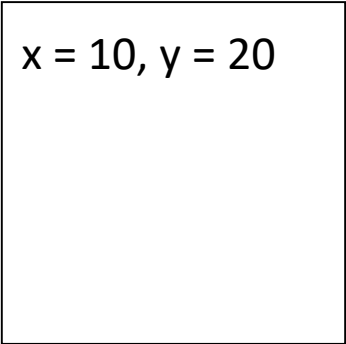
O endereço de x da main é
atribuído ao ponteiro px da
troca

O endereço de y da main é
atribuído ao ponteiro py da
troca

memória				
0x7ffe1702e370	px	0x7ffe1702e378	py	troca
0x7ffd0734b764		0x7ffd0734b768		
0x7ffd0734b764	x	0x7ffd0734b768	y	main
10		20		

```
int main(void) {  
    int x = 10, y=20;  
    printf("\nx = %d e y = %d", x, y);  
    troca(&x,&y);  
    printf("\nx = %d e y = %d", x, y);  
    return 0;  
}
```

tela



```
#include <stdio.h>
```

a variável aux é alocada e recebe o valor apontado por px

```
void troca(int *px, int *py){  
    int aux;  
    aux = *px;  
    *px = *py;  
    *py= aux;  
    return;  
}
```

memória

troca	0x7ffe1702e370 0x7ffd0734b764	px	0x7ffe1702e378 0x7ffd0734b768	py	0x7ffc2a20cf80 10	aux
	0x7ffd0734b764 10	x	0x7ffd0734b768 20	y		

```
int main(void) {  
    int x = 10, y=20;  
    printf("\nx = %d e y = %d", x, y);  
    troca(&x,&y);  
    printf("\nx = %d e y = %d", x, y);  
    return 0;  
}
```

tela

x = 10, y = 20

```
#include <stdio.h>
```

a variável aux é alocada e recebe o valor apontado por px

```
void troca(int *px, int *py){
    int aux;
    aux = *px;
    *px = *py;
    *py= aux;
    return;
}
```

memória

troca	0x7ffe1702e370 0x7ffd0734b764	px	0x7ffe1702e378 0x7ffd0734b768	py	0x7ffc2a20cf80 10	aux
main	0x7ffd0734b764 20	x	0x7ffd0734b768 20	y		

```
int main(void) {
    int x = 10, y=20;
    printf("\nx = %d e y = %d", x, y);
    troca(&x,&y);
    printf("\nx = %d e y = %d", x, y);
    return 0;
}
```

tela

x = 10, y = 20

```
#include <stdio.h>
```

a variável aux é alocada e recebe o valor apontado por px

```
void troca(int *px, int *py){
    int aux;
    aux = *px;
    *px = *py;
    *py= aux;
    return;
}
```

memória

troca	0x7ffe1702e370 0x7ffd0734b764	px	0x7ffe1702e378 0x7ffd0734b768	py	0x7ffc2a20cf80 10	aux
main	0x7ffd0734b764 20	x	0x7ffd0734b768 10	y		

```
int main(void) {
    int x = 10, y=20;
    printf("\nx = %d e y = %d", x, y);
    troca(&x,&y);
    printf("\nx = %d e y = %d", x, y);
    return 0;
}
```

tela

x = 10, y = 20

```
#include <stdio.h>
```

```
void troca(int *px, int *py){  
    int aux;  
    aux = *px;  
    *px = *py;  
    *py = aux;  
    return;  
}
```

A função troca é finalizada e o controle passa para a main para executar a próxima ação

memória

main	0x7ffd0734b764	x	0x7ffd0734b768	y
	20		10	

```
int main(void) {  
    int x = 10, y=20;  
    printf("\nx = %d e y = %d", x, y);  
    troca(&x,&y);  
    printf("\nx = %d e y = %d", x, y);  
    return 0;  
}
```

tela

x = 10, y = 20
x = 20, y = 10